

Vaskó András - Specification for the Huffman compression

1. Purpose and Scope

The aim of the program is to create a lossless compression of a given file. Huffman encoding is able to create a shorter binary codes to frequent characters and longer codes to rare ones. It does this by building a binary tree based on character frequencies. The result is a prefix-free code, meaning no code is the start of another, which makes decoding unambiguous. The encoded data can always be perfectly restored to the original.

The program will run from the command line and support two main operations:

- Encoding (compressing a text file)
- Decoding (restoring a compressed file)

2. Command-Line Usage

Usage format:

```
huffman <mode> <input_file> <output_file>
```

Where mode is:

- e Encode (compress) the input file
- d Decode (restore) the compressed file

File handling:

Before usage you have to be sure, if there's exist these files, other case you will get an error from the program.

3. Program Behaviour

3.1 Encoding (-e)

When encoding, the program will:

1. Read the text from the input file.
2. Count how many times each character occurs.
3. Generate Huffman codes: short codes for common characters, long codes for rare characters.
4. Write the output as a text-based compressed file containing:

- A code table mapping each character to its binary code
- A separator line
- The encoded data (as a stream of 0s and 1s)

3.2 Decoding (-d)

When decoding, the program will:

1. Read the code table from the compressed file.
2. Reconstruct the Huffman coding logic from this table.
3. Interpret the encoded bitstream.
4. Output the restored original text to the output file.

4. Error Handling

If the program:

- Cannot read the input file
- Cannot write the output file
- Or encounters an incorrectly formatted encoded file

Then it should:

- Print a clear error message
- Stop without crashing